PROJECTS – GIT VERSION CONTROL

Git Version Control: Unit 1 & Unit 2

General Instructions

- Study **Project 1 and Project 2**, which are provided as solved examples.
- Complete **Project 3, Project 4 and Project 5 independently**, using the solved projects as a reference.
- **You must execute the projects live in class** using Git. Merely writing the commands or copying the output will not be considered as project completion.
- All Git commands executed during the project must be **written in your Git Notebook** along with a brief explanation of each command.
- You must perform the Git operations yourself and be able to explain the purpose of the commands used.
- **Take screenshots of all major Git activities and upload them on DIGICAMPUS.**
- Screenshots must be clear and readable and should show the **Git command and its output**.
- Upload the completed project work/notebook and required screenshots in the **DIGICAMPUS assignment submission**.
- The assignment carries **100 marks**.

Important: Live Class Execution

All three assigned projects (Project 3, Project 4 and Project 5) must be executed live in class.

Unit 1 & Unit 2 Combined

Five small practical projects based on Unit A (Git Basics) and Unit B (Branching, Merging and Conflict Resolution). Projects 1 and 2 include detailed expected answers/command solutions. Projects 3–5 contain only student questions/tasks.

Topics Covered

- Repository initialization and Git configuration
- Staging, committing, status, log and diff
- Branch creation, switching and viewing
- Fast-forward and three-way merging
- Intentional merge conflicts and conflict resolution
- Tags and undo operations using checkout/restore, reset or revert

byteXL

Shoolini University

PROJECT 1 – STUDENT PORTFOLIO VERSION CONTROL

Level: Basic to Intermediate | Marks: 20

Objective

Create a small student portfolio and use Git to track its development. Demonstrate repository setup, staging, commits, history, branches, merging, a merge conflict, tagging and one undo operation.

Detailed Question / Student Task

1. Create a folder named student-portfolio and initialize it as a Git repository.
2. Configure your Git username and email and verify the configuration.
3. Create index.html, about.txt, skills.txt and projects.txt.
4. Add all files to the staging area and create the first commit with the message: Initial portfolio created.
5. Use git status, git log and git diff to inspect the repository.
6. Create an about-page branch. Add your name, course, university and a short introduction to about.txt and commit it as: About section updated.
7. Create a skills-update branch. Add programming languages and technical skills to skills.txt and commit it as: Skills section updated.
8. Switch to main and merge the feature branches. Inspect the commit history and identify the merge type.
9. Create an intentional merge conflict by changing the same line of about.txt differently on two branches. Attempt the merge and resolve the conflict manually.
10. Create a tag named v1.0 after the conflict is resolved and display the available tags.
11. Make an unnecessary change to projects.txt and demonstrate one appropriate undo operation using checkout/restore, reset or revert. Explain the result.

Answer / Expected Solution

1. Repository setup

```
mkdir student-portfolio
cd student-portfolio
git init
git config user.name "Your Name"
git config user.email "your@email.com"
git config --list
```

2. Create files and make the initial commit

```
index.html
about.txt
skills.txt
projects.txt

git status
git add .
git status
git commit -m "Initial portfolio created"
git log --oneline
```

3. Check changes

```
git diff
git diff --staged
```

git status shows the repository state; git log shows commit history; git diff shows unstaged changes; git diff --staged shows changes currently staged for commit.

4. About branch

```
git switch -c about-page
# Edit about.txt
git add about.txt
git commit -m "About section updated"
git branch
```

5. Skills branch

```
git switch main
git switch -c skills-update
# Edit skills.txt
git add skills.txt
git commit -m "Skills section updated"
git branch
```

6. Merge branches

```
git switch main
git merge about-page
git merge skills-update
git log --oneline --graph --all
```

If the target branch has not diverged, Git can perform a fast-forward merge. When both branches contain separate commits, Git may perform a three-way merge and create a merge commit.

7. Intentional merge conflict

```
# On about-page
git switch about-page
# Change the same line in about.txt to:
I am a passionate Computer Science student interested in AI.
git add about.txt
git commit -m "Updated about section for AI"

# On main, change the same line differently
git switch main
# Change it to:
I am a Computer Science student interested in software development.
git add about.txt
git commit -m "Updated about section for software development"

git merge about-page
```

Example conflict markers:

```
<<<<<<< HEAD
I am a Computer Science student interested in software development.
=======
I am a passionate Computer Science student interested in AI.
>>>>>>> about-page
```

Resolve the file, for example by combining the required information:

```
I am a Computer Science student interested in software development and AI.

git add about.txt
git commit -m "Resolved merge conflict in about section"
git status
git log --oneline --graph --all
```

8. Tag and undo

```
git tag v1.0
git tag
git log --oneline --decorate

# Make an unwanted uncommitted change to projects.txt
git status
git restore projects.txt
git status
```

The restore command removes the unwanted uncommitted modification and returns the file to its last committed state. For a committed change, git revert <commit-id> can create a new commit that reverses it.

Expected evidence

- At least five meaningful commits.
- At least two feature branches and successful merges.
- One intentionally created and resolved merge conflict.
- Tag v1.0 visible with git tag.
- One undo operation with an explanation.
- Screenshots of status, log, branch, merge/conflict resolution, tag and undo.

byteXL

Shoolini University

PROJECT 2 – COLLEGE EVENT WEBSITE

Level: Intermediate | Marks: 20

Objective

Build a small college event website using simple HTML/text files and use Git branches to manage separate features. Integrate the features, deliberately create a conflict and resolve it.

Detailed Question / Student Task

12. Create a folder named college-event and initialize it as a Git repository.
13. Configure Git username and email and verify them.
14. Create index.html, event-details.txt, schedule.txt and team.txt.
15. Create the first commit with the message: Created college event website.
16. Modify index.html and create a second commit with the message: Added event homepage.
17. Create schedule, team and design branches.
18. On schedule, add a detailed event schedule and commit it.
19. On team, add organizing team information and commit it.
20. On design, change the main event heading and commit it.
21. Merge schedule and team into main.
22. Create a deliberate conflict by changing the same heading in index.html differently on main and design, then attempt the merge.
23. Resolve the conflict, stage the file and complete the merge.
24. Create an annotated tag named v1.0-release.
25. Make an unwanted change to schedule.txt and demonstrate an appropriate undo operation.
26. Verify the final repository using git status, git branch and git log --oneline --graph --all.

Answer / Expected Solution

1. Setup and initial commit

```
mkdir college-event
cd college-event
git init
git config user.name "Your Name"
git config user.email "your@email.com"
git config --list

# Create:
index.html
event-details.txt
schedule.txt
team.txt

git add .
git commit -m "Created college event website"
git log --oneline
```

2. Homepage update

```
# Edit index.html
git diff
git add index.html
```

```
git diff --staged
git commit -m "Added event homepage"
```

3. Create feature branches

```
git branch schedule
git branch team
git branch design
git branch
```

4. Schedule and team branches

```
git switch schedule
# Edit schedule.txt
git add schedule.txt
git commit -m "Added event schedule"

git switch team
# Edit team.txt
git add team.txt
git commit -m "Added organizing team"
```

5. Merge schedule and team

```
git switch main
git merge schedule
git merge team
git log --oneline --graph --all
```

6. Create the deliberate design conflict

```
git switch main
# Change the heading in index.html to:
Annual Technology Fest 2026
git add index.html
git commit -m "Updated event heading on main"

git switch design
# Change the same heading to:
College Innovation and Technology Fest
git add index.html
git commit -m "Updated event heading on design"

git switch main
git merge design
```

7. Resolve the conflict

```
# Open index.html, remove conflict markers and keep a suitable final heading.
Annual College Innovation and Technology Fest 2026

git add index.html
git commit -m "Resolved homepage merge conflict"
git status
git log --oneline --graph --all
```

8. Annotated tag and undo

```
git tag -a v1.0-release -m "Stable college event website release"
git tag
```

```
git show v1.0-release

# Make an unwanted uncommitted change to schedule.txt
git restore schedule.txt
git status
```

The final status should be clean after the unwanted uncommitted change is restored. Students may alternatively demonstrate reset or revert on a suitable committed change and explain its effect.

byteXL

Shoolini University

PROJECT 3 – STUDENT CLUB MANAGEMENT REPOSITORY

Level: Intermediate | Marks: 20

Objective

Create a Git-managed repository for a student club. Maintain club information, events and members through separate branches and integrate the work into main.

Detailed Question / Student Task

27. Create student-club and initialize Git.
28. Configure and verify Git username and email.
29. Create README.txt, club-info.txt, events.txt and members.txt with suitable initial content.
30. Make the initial commit: Initial student club repository.
31. Update README.txt with club purpose and objectives and commit the change.
32. Create an events branch and add at least three proposed events.
33. Create a members branch and add at least five sample members.
34. Create an activities branch and add previous club activities to club-info.txt.
35. Merge events and members into main.
36. Change the same paragraph in club-info.txt differently on main and activities. Attempt the merge and create a conflict.
37. Resolve the conflict, stage the file and complete the merge.
38. Create tag v1.0.
39. Make an unwanted change to events.txt and demonstrate one undo operation.
40. Use git status, git branch and git log --oneline --graph --all to verify the repository.
41. Submit screenshots showing setup, commits, branches, merge, conflict resolution, tag and undo.

byteXL

Shoolini University

PROJECT 4 – PERSONAL EXPENSE TRACKER

Level: Intermediate | Marks: 20

Objective

Develop a small text-based expense tracker and use Git to manage categories, records and summaries through branches.

Detailed Question / Student Task

42. Create expense-tracker and initialize Git.
43. Configure Git username and email.
44. Create expense.txt, categories.txt, summary.txt and README.txt.
45. Add sample data and commit: Initial expense tracker.
46. Create a food-expenses branch and add at least five food expenses.
47. Create a travel-expenses branch and add at least five travel expenses.
48. Create a monthly-summary branch and add a monthly summary.
49. View branches and commit history before merging.
50. Merge food-expenses and travel-expenses into main.
51. Change the same line in summary.txt differently on main and monthly-summary and attempt the merge.
52. Resolve the conflict manually and complete the merge.
53. Create tag v1.0.
54. Make an incorrect change to categories.txt and demonstrate a suitable undo operation.
55. Use git diff before at least one commit and explain its output.
56. Display the final commit graph and repository status.
57. Submit the repository and screenshots for all required operations.

byteXL

Shoolini University

PROJECT 5 – MINI LIBRARY MANAGEMENT PROJECT

Level: Intermediate | Marks: 20

Objective

Create a small library-management repository and use branches for books, members and issue records while practicing merging and conflict resolution.

Detailed Question / Student Task

58. Create library-management and initialize Git.
59. Configure and verify Git username and email.
60. Create books.txt, members.txt, issued-books.txt and README.txt with suitable sample data.
61. Create the initial commit: Initial library management project.
62. Make a meaningful update to books.txt and commit it as a second version.
63. Create books-update and add at least five additional books.
64. Create members-update and add at least five sample members.
65. Create issue-records and add at least five issue/return records.
66. View branches and commit history.
67. Merge books-update and members-update into main.
68. Change the same record in issued-books.txt differently on main and issue-records. Attempt the merge.
69. Resolve the conflict without losing the required final information and complete the merge.
70. Inspect the commit graph and create tag v1.0.
71. Make an unwanted change to books.txt and demonstrate checkout/restore, reset or revert.
72. Use git status, git diff, git log and git branch for final verification.
73. Submit screenshots of repository setup, commits, branches, merge, conflict resolution, tag and undo.
74. Write a short explanation of fast-forward versus three-way merge based on your project.